

1. Resumen

Este proyecto implementa un simulador de planificación de procesos en C++17 que modela y compara dos algoritmos de scheduling: First Come First Served (FCFS) y Round Robin (RR) con quantum configurable. El simulador lee cargas sintéticas de procesos desde archivos de texto (formato CSV: id, llegada, ráfaga), calcula tiempo de espera, tiempo de retorno y utilización de CPU por proceso y en promedio, y genera un diagrama de Gantt en formato ASCII. Se utilizaron dos conjuntos de datos: uno de 10 procesos (requisito mínimo del enunciado) y una ampliación voluntaria de 50 procesos generados con semilla fija para garantizar reproducibilidad. Los resultados muestran que FCFS produce el menor tiempo de espera promedio en ambos datasets (17.40 en 10 procesos, 180.72 en 50 procesos), frente a Round Robin con quantum=2 (22.10 y 251.96) y quantum=4 (20.70 y 247.10). Se observa que, al aumentar el quantum, Round Robin se aproxima progresivamente al comportamiento de FCFS. La utilización de CPU fue del 100% con el dataset de 10 procesos y del 99.78% con el de 50, diferencia explicada por un breve instante de inactividad inicial antes de la primera llegada. Los resultados coinciden con el comportamiento teórico descrito en la bibliografía del curso.

Palabras clave: scheduling, FCFS, Round Robin, planificación de procesos, diagrama de Gantt, C++17

2. Introducción

2.1 Contexto

La planificación de procesos (scheduling) es uno de los mecanismos centrales de todo sistema operativo multiprogramado: decide qué proceso en estado listo recibe la CPU en cada instante, y esa decisión determina directamente métricas como el tiempo de espera, el tiempo de retorno y la utilización de la CPU. Comprender cómo se comportan distintos algoritmos de planificación bajo la misma carga de trabajo es esencial para razonar sobre el diseño de un sistema operativo.

2.2 Objetivo general

Implementar un simulador en C++ que modele los algoritmos FCFS y Round Robin sobre una carga de procesos sintética, permitiendo comparar cuantitativamente su comportamiento.

2.3 Objetivos específicos

1. Diseñar una estructura de datos que represente un proceso con sus datos de entrada y sus métricas calculadas.
2. Implementar FCFS con ordenamiento estricto por tiempo de llegada, sin apropiación.
3. Implementar Round Robin con quantum configurable como parámetro de entrada.
4. Calcular y reportar tiempo de espera y tiempo de retorno por proceso y en promedio, para ambos algoritmos.

5. Calcular la utilización de CPU (% de tiempo no ocioso) durante la simulación.
6. Generar un diagrama de Gantt en formato ASCII que muestre el orden real de ejecución.
7. Comparar cuantitativamente ambos algoritmos con al menos dos valores distintos de quantum.

2.4 Alcance y limitaciones

El simulador asume que todos los procesos son independientes, que no hay operaciones de entrada/salida durante la ráfaga de CPU, y que el costo del cambio de contexto es despreciable. No se implementan algoritmos con prioridades, Shortest Job First (SJF), ni colas multinivel — quedan fuera del alcance de este proyecto. La interfaz es de consola, sin elementos gráficos adicionales.

3. Marco Teórico

3.1 Fundamentos de planificación de procesos

La planificación de procesos es la actividad del sistema operativo que determina cuál de los procesos en estado listo será ejecutado por la CPU. El planificador de corto plazo (short-term scheduler) se ejecuta con alta frecuencia buscando maximizar la utilización de la CPU y minimizar el tiempo de espera de los procesos [1]. Las métricas centrales para evaluar un planificador son el tiempo de espera (waiting time), el tiempo de retorno (turnaround time) y la utilización de CPU.

3.2 First Come First Served (FCFS)

FCFS es el algoritmo de planificación más simple: el proceso que llega primero es el primero en ser atendido, mediante una cola FIFO. Es no apropiativo (non-preemptive): una vez que un proceso obtiene la CPU, la conserva hasta finalizar su ráfaga completa. Su principal desventaja teórica es el efecto convoy, en el que procesos con ráfagas cortas quedan esperando detrás de procesos con ráfagas largas [1][2].

3.3 Round Robin (RR)

Round Robin es un algoritmo apropiativo (preemptive) pensado para sistemas de tiempo compartido. Cada proceso recibe un quantum fijo de CPU; si no termina su ráfaga dentro de ese quantum, es interrumpido y regresa al final de la cola de listos. El tamaño del quantum es un parámetro crítico: valores pequeños aumentan la equidad entre procesos pero también el número de cambios de contexto, mientras que valores grandes hacen que Round Robin se aproxime al comportamiento de FCFS [2][3].

3.4 Relación con xv6

El kernel de xv6 implementa una forma de planificación round-robin en su función `scheduler()` (`proc.c`), recorriendo la tabla de procesos en busca del siguiente proceso en estado `RUNNABLE`. A diferencia de xv6 —que recorre un arreglo fijo de procesos— este simulador utiliza una cola explícita (`std::queue<int>`) de índices para representar la cola de listos, lo que permite manejar de forma más directa la llegada de nuevos procesos durante la simulación y el reingreso de procesos interrumpidos.

4. Diseño de la Solución

4.1 Arquitectura general

El simulador se organiza en módulos con responsabilidades separadas: una capa de datos (**Proceso**), una capa de entrada/salida (**Lector**), una capa de algoritmos (**FCFS**, **RoundRobin**), y una capa de reporte (**Metricas**, **Gantt**), coordinadas por **main**. Esta separación responde al requisito no funcional de modularidad del enunciado: la lógica de simulación nunca depende de cómo se leyeron los datos, y ambos algoritmos comparten el mismo tipo de resultado.

4.2 Estructuras de datos

La estructura **Proceso** (Fragmento 1) separa los datos de entrada (**id**, **llegada**, **rafaga**, que nunca se modifican tras la lectura) de los datos calculados durante la simulación (**restante**, **finalizacion**, **tiempoEspera**, **tiempoRetorno**). El campo **restante** es una copia de trabajo de **rafaga** que Round Robin va descontando en cada quantum, sin alterar el dato original.

```
struct Proceso {
    int id;
    int llegada;
    int rafaga;
    int restante;
    int finalizacion;
    int tiempoEspera;
    int tiempoRetorno;

    Proceso(int id_, int llegada_, int rafaga_)
        : id(id_), llegada(llegada_), rafaga(rafaga_),
          restante(rafaga_), finalizacion(0),
          tiempoEspera(0), tiempoRetorno(0) {}
};
```

Fragmento 1: Estructura del proceso.

Para la cola de listos de Round Robin se utiliza `std::queue<int>`, almacenando índices dentro del vector de procesos en lugar de copias de la estructura completa — una cola es la estructura de datos naturalmente apropiada porque Round Robin es, por definición, un reparto de turnos en orden FIFO con reingreso al final.

Ambos algoritmos devuelven un **ResultadoSimulacion** común (vector de procesos con tiempos calculados, más una traza de **Segmentos** de ejecución), lo que permite que **Metricas** y **Gantt** funcionen sin conocer qué algoritmo produjo el resultado.

4.3 Supuestos de diseño

- Los procesos se leen del archivo en cualquier orden; ambos algoritmos los reordenan internamente por tiempo de llegada.
- No hay operaciones de E/S durante la ráfaga de CPU.
- El cambio de contexto tiene costo cero.
- Todos los procesos están listos para ejecutar en cuanto llegan (no hay bloqueos).

5. Implementación

5.1 Lenguaje y estándar

C++17, biblioteca estándar únicamente (sin dependencias externas). Compilado y probado con g++ 15.2.0 sobre Ubuntu 26.04 LTS. El código no utiliza ninguna característica específica de plataforma, por lo que se espera que compile igual en Windows (MinGW o MSVC) sin modificaciones.

5.2 Módulos principales

- **Proceso.h**: estructura de datos del proceso (ver Fragmento 1).
- **Lector.h/.cpp**: parser del archivo de entrada (formato `id,llegada,rafaga`), con manejo de errores línea por línea — líneas malformadas o con valores inválidos se reportan por `stderr` y se descartan sin abortar la lectura completa.
- **FCFS.h/.cpp**: ordena los procesos por llegada y los ejecuta secuencialmente, calculando inicio, fin, espera y retorno de forma directa (no hay apropiación).
- **RoundRobin.h/.cpp**: mantiene una cola de listos (`std::queue<int>`) y un quantum configurable; cada proceso ejecuta como máximo `min(quantum, restante)` unidades por turno.
- **Metrics.h/.cpp**: calcula promedios de espera/retorno y el % de utilización de CPU a partir de la traza de segmentos.
- **Gantt.h/.cpp**: construye la representación ASCII del orden de ejecución a partir de la misma traza de segmentos.

5.3 Compilación y ejecución

`make`

```
./simulador data/procesos.txt 2  
./simulador data/procesos.txt 4
```

6. Resultados Experimentales

6.1 Entorno de prueba

Máquina virtual Ubuntu 26.04 LTS (VirtualBox), compilador g++ 15.2.0, flags `-std=c++17 -Wall`. Dado que el simulador modela tiempo lógico de eventos discretos (no tiempo real de ejecución), las métricas de espera/retorno/utilización no dependen del hardware del host; se midió adicionalmente el tiempo real de ejecución del programa como referencia de rendimiento: 0.135s para el dataset de 10 procesos y 0.077s para el de 50 procesos (tiempo *real* de `time`, dominado por el arranque del proceso, no por el cálculo en sí).

6.2 Dataset de 10 procesos

Dataset mínimo requerido por el enunciado: 10 procesos con llegadas entre 0 y 9, ráfagas entre 2 y 8.

Tabla 1: Comparativa de métricas promedio (10 procesos)

Métrica	FCFS	RR (q=2)	RR (q=4)
Tiempo de espera promedio	17.40	22.10	20.70

Métrica	FCFS	RR (q=2)	RR (q=4)
Tiempo de retorno promedio	21.90	26.60	25.20
Utilización de CPU	100.00%	100.00%	100.00%

6.3 Dataset extendido de 50 procesos

Ampliación voluntaria (no exigida por el enunciado), generada con semilla fija 20260729 para garantizar reproducibilidad: llegadas acumuladas con saltos de 0 a 3 unidades, ráfagas uniformes entre 1 y 15.

Tabla 2: Comparativa de métricas promedio (50 procesos)

Métrica	FCFS	RR (q=2)	RR (q=4)
Tiempo de espera promedio	180.72	251.96	247.10
Tiempo de retorno promedio	189.96	261.20	256.34
Utilización de CPU	99.78%	99.78%	99.78%

6.4 Diagrama de Gantt

La Figura 1 muestra el diagrama de Gantt ASCII generado para FCFS con el dataset de 10 procesos.

```
| P1 | P2 | P3 | P4 | P5 | P6 | P7 | P8 | P9 | P10 |
0   5   8  16  22  24  28  35  38  43  45
```

Figura 1: Diagrama de Gantt para FCFS, dataset de 10 procesos.

6.5 Comparación cuantitativa

En ambos datasets, FCFS obtiene el menor tiempo de espera y de retorno promedio. Al aumentar el quantum de Round Robin de 2 a 4, el tiempo de espera promedio se acerca al de FCFS (de 22.10 a 20.70 en el dataset de 10 procesos; de 251.96 a 247.10 en el de 50), confirmando que un quantum mayor aproxima Round Robin al comportamiento de FCFS. La utilización de CPU solo bajó del 100% en el dataset de 50 procesos, por un breve instante de inactividad antes de la llegada del primer proceso.

7. Evaluación ISO/IEC 25010

Característica	Calificación	Justificación
Rendimiento	4	Se midió el tiempo real de ejecución del programa (0.135s con 10 procesos, 0.077s con 50), y se comparó cuantitativamente el efecto del quantum sobre el tiempo de espera/retorno en dos datasets de tamaño distinto (Tablas 1 y 2).
Fiabilidad	4	El lector de archivos (<code>Lector.cpp</code>) maneja explícitamente archivos inexistentes (excepción), líneas con campos faltantes, valores no numéricos y valores semánticamente inválidos (llegada negativa, ráfaga menor o igual a 0), sin abortar la lectura completa. El dataset de 50 procesos se genera con semilla fija, garantizando resultados reproducibles.
Usabilidad	3	La interfaz de consola reporta uso correcto de argumentos (<code>Uso: simulador <archivo> <quantum></code>) y mensajes de error legibles, pero no tiene ayuda interactiva ni colores — es funcional pero mínima.
Compatibilidad	3	El código usa exclusivamente la biblioteca estándar de C++17, sin llamadas específicas de Linux; se espera que compile igual en Windows, pero esto no fue verificado directamente en esa plataforma (solo se probó en Ubuntu 26.04 LTS).

Característica	Calificación	Justificación
Mantenibilidad	4	Código separado en 6 módulos con una única responsabilidad cada uno (Proceso, Lector, FCFS, RoundRobin, Metricas, Gantt), nombres descriptivos en español, y un tipo de resultado compartido (<code>ResultadoSimulacion</code>) que evita duplicar lógica entre algoritmos.

8. Discusión de Resultados

8.1 Interpretación de resultados

Los resultados confirman el comportamiento teórico esperado: FCFS minimiza el tiempo de espera promedio en ambos datasets porque nunca interrumpe un proceso en ejecución, evitando el overhead de cambios de contexto que sí introduce Round Robin. La diferencia se amplía con más procesos: en el dataset de 50, FCFS tiene un tiempo de espera 28.2% menor que RR $q=2$ y 26.9% menor que RR $q=4$.

8.2 Relación con la teoría

Los resultados coinciden con lo descrito en la bibliografía del curso [1][2][3]: FCFS produce el menor tiempo de espera cuando no hay un efecto convoy severo (en este dataset sintético las ráfagas son relativamente parejas); Round Robin con quantum pequeño reparte la CPU de forma más equitativa pero a costa de mayor tiempo de retorno; y un quantum más grande en Round Robin acerca su comportamiento al de FCFS, tal como predice la teoría [3].

8.3 Limitaciones encontradas

- El simulador no modela el costo del cambio de contexto, lo que favorece artificialmente a Round Robin frente a un sistema real.
- No se modelan operaciones de entrada/salida durante la ráfaga, por lo que la utilización de CPU es siempre cercana al 100%.
- La compatibilidad con Windows no fue verificada directamente, solo se argumenta por ausencia de dependencias de plataforma.

9. Conclusiones

Se implementó un simulador de scheduling que compara cuantitativamente FCFS y Round Robin bajo cargas sintéticas controladas, cumpliendo todos los requisitos funcionales y no funcionales

especificados en el enunciado. FCFS mostró consistentemente menor tiempo de espera y retorno promedio en ambos datasets probados, mientras que Round Robin ofrece un reparto más equitativo de la CPU entre procesos, a costa de mayor tiempo de retorno.

Aprendizajes clave: - El tamaño del quantum en Round Robin es el parámetro que más influye en su cercanía al comportamiento de FCFS. - Separar la lógica de simulación (algoritmos) de la lógica de entrada/salida (lector) y de reporte (métricas, Gantt) permite que ambos algoritmos compartan el mismo código de post-procesamiento sin duplicación. - Diseñar la estructura de datos pensando en qué campos son “de entrada” y cuáles son “calculados” simplifica considerablemente el cálculo de métricas derivadas.

Posibles extensiones

- Implementar Shortest Job First (SJF) y planificación por prioridades para comparar contra FCFS/RR.
 - Modelar el costo del cambio de contexto como parámetro configurable.
 - Verificar la compatibilidad multiplataforma compilando directamente en Windows.
-

10. Referencias

- [1] Silberschatz, A., Galvin, P. B., & Gagne, G. (2018). *Operating System Concepts* (10a ed.). Wiley.
- [2] Tanenbaum, A. S. (2014). *Modern Operating Systems* (4a ed.). Pearson.
- [3] Arpaci-Dusseau, R. & Arpaci-Dusseau, A. (2018). *Operating Systems: Three Easy Pieces*. Arpaci-Dusseau Books.